THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 04

Legacy / Mainframe Modernization

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

01



YOUR STUDY COMPANION

Solution architecture case study • Financial-services account servicing • 3 November 2026 Design position: Incremental, domain-by-domain modernization with explicit ownership, compatibility layers, verified data replication, safe cutover, and a viable rollback plan.

> **Scenario, not a claim about American Express:** A large card issuer has a hypothetical IBM z/OS estate using COBOL/CICS and Db2 for z/OS, with batch jobs and several dependent channels. Customers need account profile, transaction-history, and address-change capabilities. The existing platform is business-critical. Actual technology, SLOs, volumes, compliance boundaries, and downstream dependencies must be discovered.

NAVIGATION

Inside this guide

01	1. Executive design decision
02	2. Problem, scope, and discovery
03	3. Quality requirements and their architectural consequences
04	4. Baseline: typical legacy architecture
05	5. Recommended coexistence architecture
06	6. Derive the main architectural choices
07	7. API and data contracts
08	8. Critical request flows
09	9. Data migration, ordering and correctness
10	10. Migration roadmap with entry / exit gates
11	11. Failure handling and resilience
12	12. Security and compliance architecture
13	13. Testing and cutover proof
14	14. Observability, governance, cost, and ownership
15	15. Alternatives, objections, and decision triggers
16	16. Interview-ready narrative
17	17. Official technical references

CHAPTER

1. Executive design decision

Choose the Strangler Fig pattern, not a one-shot rewrite. Introduce a stable API facade and an anti-corruption layer (ACL) around existing business transactions. Extract **transaction-history reads first**, using a replicated read projection where the source supports committed-change capture. Keep **address-change writes on the mainframe** until the entire bounded context and its dependent consumers can be transferred safely. Move write ownership only once, with a controlled cutover, reconciliation, and explicitly engineered rollback path.

Why: a mainframe is not automatically slow or unreliable. The problem is usually *change coupling*, access to modern interfaces, undocumented dependencies, or cost/skills—not age alone. Preserving working transaction logic reduces the initial blast radius.

Hard invariants:

1. For each business record/domain, exactly one system owns authoritative writes at a time.
2. Never claim success for a write unless the authoritative transaction committed or an explicit asynchronous contract says *accepted, pending*.
3. Treat CDC projections as eventually consistent, not as a second transactional source of truth.
4. Never retry an unknown-outcome write blindly; first establish whether the authoritative operation committed.
5. Retirement of legacy code is optional: retain shared core logic if risk and economics favor it.

CHAPTER

2. Problem, scope, and discovery

Business problem

- Faster, independently deployable digital features without destabilizing card/account servicing.
- Consistent interfaces for mobile, web, and partner channels.
- Reduced coupling to COBOL copybooks, CICS transaction interfaces, batch windows, and hidden dependencies.
- Verifiable continuity of customer data, business rules, security, and auditability during migration.

In-scope use cases

- Read account profile and recent transaction history.
- Change an account's mailing address with authorization, concurrency checks, and audit trail.
- Continue running required nightly jobs and legacy downstream integrations during coexistence.
- Expose a stable versioned API while internal implementations change.

Out of scope for this first wave: replacing the card authorization switch, general ledger, all batch processing, and all mainframe databases. Those need separate domain-level designs and risk assessments.

Discovery before selecting technology

Inventory COBOL programs, CICS/IMS transactions, Db2 tables, VSAM files, copybooks, IBM MQ integrations, JCL/batch dependencies, schedulers, security identities, report consumers, transaction boundaries, data retention, code-page and packed-decimal formats, and business rule owners. Trace a *business capability* through online and batch code; do not infer boundaries from table names alone. Find which consumers read data directly or bypass intended APIs.

Sizing: Request actual read/write peak throughput, concurrent sessions, mainframe transaction capacity, batch-period peaks, p95/p99 latency baselines, dataset growth, network RTT, maximum tolerable replica lag, and recovery targets. Do not invent SLO numbers. For an illustrative capacity model, if peak reads = `R` requests/s and each request causes `C` mainframe calls, legacy demand is approximately `R × C`; reducing `C` by serving eligible reads from a projection is a primary reason for read extraction. Validate that replication cost does not erase this gain.

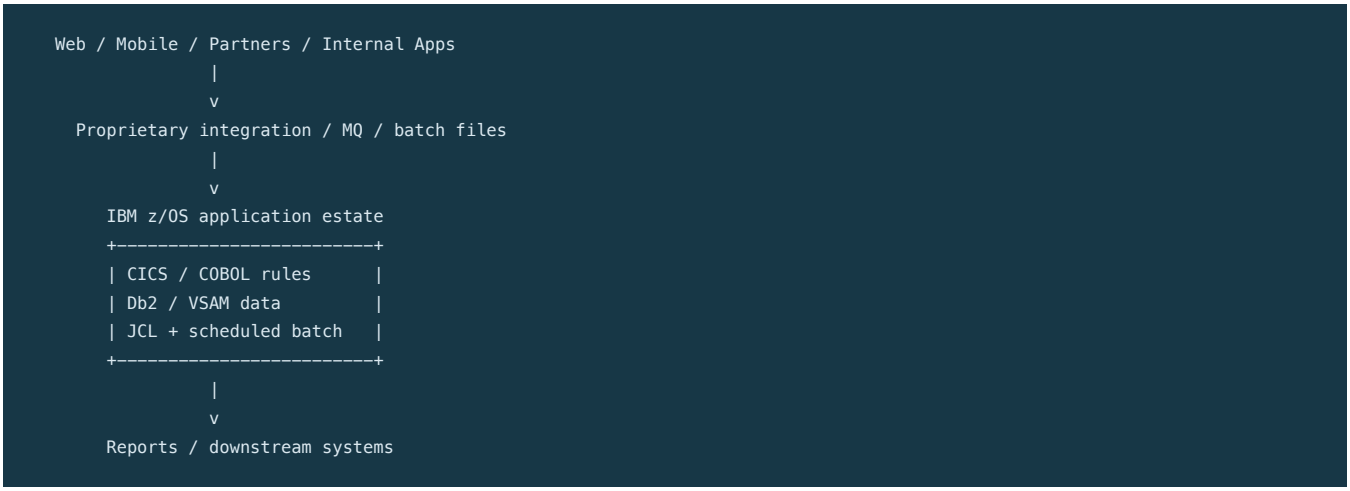
CHAPTER

3. Quality requirements and their architectural consequences

Requirement	Design consequence	Why
Correct address changes	Single authoritative writer + transactional legacy call + idempotency	Avoid divergent addresses and duplicate side effects
Responsive transaction history	Dedicated read projection with bounded freshness	Avoid repeated cross-network, high-volume OLTP calls
No business-rule regression	Preserve existing rules initially; golden-master comparisons	Hidden rules may reside in COBOL, JCL, and downstream jobs
High availability	Isolate channels from mainframe failures; defined degradation	Digital frontend must not amplify a core outage
Audit and privacy	AuthN/AuthZ, purpose-based access, field minimization, immutable trails	Financial data has elevated risk
Controlled migration	Route by capability, cohort, and ownership	Small reversible read rollout; explicit write cutover
Cost efficiency	Avoid chatty integration; measure capacity and run cost	New cloud components plus old platform can temporarily cost more

CHAPTER

4. Baseline: typical legacy architecture



The exact stack above is a **hypothesis to validate**. Some estates already have modern APIs, events, and strong operational separation. The modernization plan should reuse them rather than duplicate them.

CHAPTER

5. Recommended coexistence architecture





Placement of components:

- **Facade / API gateway:** Stable public contract, authentication enforcement, rate limits, routing, versioning. It does not implement core business rules.
- **Capability router:** A controlled map of which backend owns *each* operation. A flag changes routing; it must not change source-of-truth ownership by itself.
- **ACL:** Converts modern domain terms into legacy request records, statuses, data encodings, and error semantics. Prevents copybook and COBOL concerns from leaking into digital services.
- **Mainframe adapter:** Use an approved integration such as IBM z/OS Connect for supported REST exposure of CICS/IMS/Db2 assets; IBM MQ may be a better fit for existing asynchronous interfaces. Choice depends on proven availability and transaction semantics.
- **Read projection:** A database optimized for transaction-history queries. It is not authoritative and must expose its freshness and replay/rebuild mechanism.
- **CDC stream:** Copy committed mainframe changes where source and tooling allow, maintain checkpoint/watermark, and support replay, DLQ, and gap detection. Check whether data lives in Db2 versus VSAM or batch files before choosing CDC technology.
- **Existing batch:** Remains part of the truth model until each job and every report/consumer is explicitly migrated or retired.

CHAPTER

6. Derive the main architectural choices

Decision	Selected approach	Why not the obvious alternative?	Revisit when…
Migration strategy	Incremental strangler	Big-bang rewrite combines unknown business rules, migration, and cutover risk	Capability is isolated, fully specified, and cheaply replaceable

Preserve core logic	Expose legacy transaction through ACL	Reimplementing COBOL rules first risks subtle functional differences	Rules are covered by parity tests and ownership is transferable
Service boundaries	Business capability (history, account servicing)	One microservice per COBOL program merely recreates legacy coupling	Domain events, ownership, and dependencies are understood
Read migration	Snapshot + CDC + read projection	Requerying Db2 for every digital request couples traffic to mainframe	Read freshness must be strictly transactional
Write migration	One writer; mainframe first	Dual writes produce inconsistent state on partial failure	Whole write domain is cut over with reliable synchronization
Sync/async	Sync for address-command result; async for read projections and reporting	Pure async cannot promise a completed write immediately	API deliberately offers accepted/pending workflow
Data store	Relational store for operation tracking; query-shaped projection for reads	NoSQL is not intrinsically faster or safer for every workload	Demonstrated scale and access patterns justify a different model
Hosting	Privately connected, managed application runtime + managed persistence	Direct browser-to-mainframe exposure has poor isolation and security	Measured operating requirements favor another runtime
Event delivery	At-least-once + idempotent consumers	'Exactly once' across heterogeneous systems cannot be assumed	A specific connector and entire transactional boundary prove stronger guarantees

Key vocabulary: *Strangler Fig* = replace functionality piece by piece behind a stable interface. *ACL* = a translating adapter between incompatible models. *Bounded context* = a domain with coherent rules and ownership. *CDC* = replicate committed database changes. *Idempotency* = repeating an operation identifier does not repeat the business effect. *System of record* = the authority for a particular data domain.

CHAPTER

7. API and data contracts

Use business-oriented APIs rather than raw COBOL transaction names:

```
GET /v1/accounts/{accountId}/transactions?cursor=...&limit=50
Authorization: Bearer <access-token>

200 OK
{
  "items": [{"transactionId":"t-123","amountMinor":2750,
             "currency":"USD","postedAt":"..."}],
  "nextCursor":"...", "dataAsOf":"..."
}
```

`dataAsOf` documents replica freshness. The read query uses deterministic pagination; do not promise the most recent write is visible immediately in an asynchronous projection.

```
PUT /v1/accounts/{accountId}/mailing-address
Authorization: Bearer <access-token>
Idempotency-Key: client-operation-uuid
If-Match: "account-version"

{ "address": {"line1":"...","city":"...", "region":"...","postalCode":"..."} }
```

Return `200` only after an authoritative commit and verified outcome. Return `409` or `412` for an account-version conflict under the documented API contract. If the outcome is unknown after a timeout, return a documented pending/unknown-operation response with `operationId` and a status endpoint—**not a false failure or a second write attempt**.

Minimal data ownership model

Entity / record	Authority during coexistence	Modern copy / use
Account profile	Mainframe	Optional read cache or projection
Mailing address	Mainframe	Non-authoritative display; no independent write
Posted transaction history	Mainframe	CDC-built read projection
Operation idempotency record	Command integration boundary plus authoritative dedup support	Status/retry control; needs end-to-end design
Routing ownership registry	Modern platform governance	Versioned and audited capability routing
Audit events	Approved audit system	Append-only security/change evidence

Encoding and semantics: define canonical account identifiers; currency in integer minor units or exact decimals; ISO dates and explicit timezone; code-page conversion; COBOL COMP-3/packed decimals if present; null/blank semantics; and error-code translation. Never change rounding or business calendar rules silently.

CHAPTER

8. Critical request flows

A. Transaction-history read (lower-risk first migration)

1. Client authenticates; gateway enforces account authorization and input limits.
2. Router selects the modern History Read Service for eligible cohorts.
3. Service queries its indexed projection by `accountId`, timestamp, and deterministic cursor; returns freshness watermark.
4. Projection consumer applies **committed** CDC events, deduplicates, and records source checkpoints.
5. If the projection is stale beyond the agreed freshness policy, return explicit stale/error behavior or use a tightly controlled legacy fallback. **Never silently claim real-time accuracy.**

Why this order: extracted reads are comparatively reversible. The legacy mainframe retains write ownership. A projection can be rebuilt from a consistent snapshot and durable change log; if this cannot be guaranteed, don't market it as lossless CDC.

B. Mailing-address change (initially legacy-owned)

1. Authorize the authenticated user for the specific account and verify the expected profile version.
2. Persist a client operation key and immutable request hash. Duplicate key + different payload is rejected.
3. Call the ACL using a stable `operationId`; ACL maps canonical address to legacy schema and validates encoding.
4. Mainframe CICS transaction performs the authoritative change **and ideally records the same operation identifier atomically**. Return a committed status and new version.
5. If commit is confirmed, complete the modern operation record and respond success. CDC later updates read copies.
6. If the network times out, record `UNKNOWN` and query status/reconcile. Do **not** blindly replay. If legacy cannot check/deduplicate operation IDs, automatic retries of writes must remain disabled until a safe protocol exists.

Why a local idempotency table is insufficient: a crash after the mainframe commits but before the modern service records success leaves an ambiguous outcome. Deduplication or status inquiry must cross the legacy transaction boundary.

C. Batch completion / CDC reconciliation

1. Discover whether overnight jobs mutate Db2, VSAM, generated files, and derived totals.
2. If Db2 CDC observes those *committed* mutations, publish into the history projection with source positions.
3. For file- or VSAM-derived changes outside CDC coverage, create a separate governed change feed or scheduled snapshot/delta ingestion.
4. Compare key counts, record-level hashes/fields, business totals, missing IDs, and source-versus-target watermarks.
5. Quarantine mapping defects, emit alerts, and rerun a safe replay/rebuild without changing mainframe truth.

CHAPTER

9. Data migration, ordering and correctness

Bootstrap: Capture a consistent source snapshot and a matching source-log position. Load the snapshot, then apply all subsequent committed changes from that position. Do not take a snapshot and start CDC later without overlap controls: a gap can lose mutations. Use connector-specific supported bootstrap methods and verify recovery behavior.

Event correctness: Carry `sourceRecordId`, `sourceCommitPosition`, schema version, operation type, and event ID. Use idempotent upserts and deletes. Preserve per-record ordering using source log order/sequence checks; a distributed event bus does not magically guarantee global order. Invalid events go to quarantine/DLQ with replay tooling.

Read consistency: Transaction history is eventually consistent; address updates are acknowledged only by the authoritative writer. If a workflow requires read-your-writes consistency, route that read to the authoritative source or use a version barrier/wait-for-projection policy. Don't use stale cached data to decide whether an account can be changed.

Write ownership transfer (later wave): Freeze or drain incoming commands for the bounded context, complete source-to-target catch-up, reconcile invariants, switch the *single writer* routing, restart commands against the new owner, and monitor with an agreed rollback window. Once new writes occur, reversal requires a trustworthy reverse sync or forward-recovery plan; changing a feature flag alone is not a complete rollback.

CHAPTER

10. Migration roadmap with entry / exit gates

Phase	Scope	Gate to advance	Rollback reality
0. Assess	Program/dependency map, data lineage, workloads, security	Identified owners, interfaces, baselines, risk register	No customer impact
1. Wrap	Facade, contract tests, observability; 100% legacy backing	Same outputs, approved access controls, capacity protection	Route back to existing entry point
2. Replicate	Snapshot, CDC, read projection; shadow reads only	Coverage and reconciliation reliable; freshness acceptable	Stop serving projection
3. Route reads	Progressive eligible-cohort rollout	Parity, error rate, capacity, and lag meet agreed thresholds	Route reads to original source if capacity permits
4. Modern commands	New command API, still legacy-owned write	Idempotency across boundary, status inquiry, audit parity	Route commands through previous legacy path
5. Transfer domain	New owner of a *complete, bounded*	Cutover validation + all dependent	Requires reverse-sync/forward-fix

	write domain	consumers changed	design
6. Retire selectively	Remove unused programs, files, interfaces	No hidden callers, jobs, or regulatory retention blockers	Preserve backup/archival access as required

How to pick the first domain: Prefer high business value, clear ownership, low coupling, stable rules, testable outputs, and tolerable eventual consistency. Transaction-history search is an illustration, not automatically suitable for every organization.

CHAPTER

11. Failure handling and resilience

Failure	Response / mitigation
Mainframe unavailable for a command	Fail closed for writes or expose an explicitly durable pending workflow if business rules allow; never fabricate commit
CICS call times out after commit	Store `UNKNOWN`, query using operation ID, reconcile; prevent unsafe retry
CDC falls behind	Alarm on lag and source positions; return `dataAsOf`; suspend stale-sensitive features
CDC event duplicated/out of order	Idempotent consumer, per-record source sequence, replay/DLQ
Legacy capacity threatened by fallback	Per-operation rate limits, bulkhead pool, circuit breaker; shed low-priority reads
New read store outage	Controlled legacy fallback only with tested capacity; otherwise explicit degradation
Network partition	Avoid split-brain writes; current owner remains sole writer
Batch job changes hidden state	Inventory jobs; compare end-of-batch totals and source-to-target state
Feature rollback after modern writes	Not safe without reverse data transfer and reconciliation; favor forward recovery when necessary

Retries: safe reads may use bounded retries with jitter if allowed. Writes may only retry when end-to-end idempotency and unknown-outcome lookup have been proven. Set dependency timeouts within the caller's deadline and use circuit breakers to protect z/OS capacity.

CHAPTER

12. Security and compliance architecture

- **Identity:** OIDC/OAuth2 for channels when appropriate; service-to-service identity with mTLS or signed workload credentials. Map users and technical identities to approved mainframe authorization mechanisms.
- **Authorization:** Account-level entitlements on every request. Gateway authentication alone is insufficient. Ensure least-privilege service accounts and separation of duties.
- **Network:** Private connectivity, segmentation, ingress/egress allowlists, certificate and key rotation, monitored trust boundaries. Do not expose raw CICS/Db2 interfaces to public clients.
- **Data:** Minimize replicated fields; tokenize or omit cardholder data where possible, encrypt in transit and at rest, control KMS/HSM access, remove secrets/PII from logs and dead-letter payloads. Determine actual PCI DSS scope with the responsible security/compliance team rather than assuming every component is out of scope.
- **Audit:** Record who requested the change, authorization decision, operation ID, before/after references where policy permits, mainframe result, routing policy version, and reconciliation outcomes. Apply retention and tamper-evidence requirements.

- **Threat cases:** spoofed caller, replayed request, compromised adapter, unintended CDC overexposure, stale authorization, privilege escalation, and data exfiltration.

CHAPTER

13. Testing and cutover proof

Functional parity: Build a *golden-master* suite by capturing approved, sanitized legacy inputs and expected outputs. Compare business outcomes across dates, currencies, holidays, error codes, missing addresses, batch cutoffs, duplicate requests, and COBOL data representations. Shadow **reads** are safe; shadow **writes against production** are not, unless designed as side-effect-free validation.

Migration checks: Count and hash records at defined source positions; compare per-account history and totals; verify CDC initialization without gaps; test delete handling, schema changes, replay, backfill and recovery from checkpoint loss.

Non-functional: Performance tests include network latency, CICS concurrency and peak batch contention, connection pool saturation, burst traffic, dependency failure, security policy, penetration testing, and recovery drills. Test version/contract compatibility against all consumers, including forgotten internal and batch clients.

Cutover runbook: Freeze requirements and mappings → validate baseline/checkpoints → confirm downstream readiness → canary read cohorts → reconcile → publish routing change → monitor → decide continue/stop based on pre-agreed gates. For writes, add command drain/freeze and reverse-sync/forward-fix procedures before any switch.

CHAPTER

14. Observability, governance, cost, and ownership

Metrics: End-to-end p50/p95/p99 latency, error rate, dependency timeout, CICS calls per request, queue age, CDC lag and position gaps, projection freshness, data mismatch count, unknown-outcome commands, batch duration, routing cohorts, and manual reconciliations. Correlate modern trace IDs with legacy request/transaction identifiers where feasible; do not place sensitive payloads in traces.

Operations: Named owners for gateway, ACL, mainframe service, CDC, projection, schema mapping, and reconciliation. Maintain an architecture decision record (ADR), data ownership register, contract versions, operational handoffs, incident playbooks, and service dependency map.

Cost model: Compare legacy CPU/transaction demand and software charges, connectivity, CDC licensing, storage, managed compute, security operations, team cost, and coexistence overhead. The temporary cost of running *both systems* is a real migration cost; do not promise automatic savings.

CHAPTER

15. Alternatives, objections, and decision triggers

Alternative	Appropriate when	Main reason not to select by default
Big-bang rewrite	Small self-contained application, fully understood rules, controlled outage	High correlated business/data/cutover risk
Replatform COBOL unchanged	Strong need to leave current hardware; preserve tested logic	Moves runtime but does not remove coupling or fix domain design
Automated refactoring	Proven conversion tooling and parity tests, stable domain model	Generated code may preserve structural debt; hard to prove equivalence for every rule
Retain mainframe + modern APIs	Core remains efficient, stable, compliant, and economically sensible	Does not solve every long-term agility constraint

CDC + CQRS reads	Stale-tolerant read workload and reliable committed-change feed	Extra state, monitoring, and reconciliation complexity
Synchronous all the way to mainframe	Low traffic, strict freshness, no reliable CDC	Higher latency and mainframe dependency per request
Bidirectional synchronization	Only with fully specified conflict-resolution/ownership invariants	Split-brain risk and very complex recovery

Architectural principle: A domain can be *modernized* without immediately leaving z/OS. “Cloud-first” is not a sufficient business reason to migrate stateful financial transaction logic.

CHAPTER

16. Interview-ready narrative

> “I would not begin by rewriting COBOL. I would first map business capabilities, transaction boundaries, batch dependencies, and ownership. Then I would introduce a stable API facade with an anti-corruption layer that calls the existing CICS/Db2-backed functions. I would move lower-risk reads first using a consistent snapshot and CDC-based projection, with explicit freshness and reconciliation. Writes would continue to have one source of truth until an entire bounded context could be cut over. The difficult decisions are idempotency across the legacy boundary, unknown commit outcomes, CDC coverage, batch coupling, data parity, and genuine rollback after writer ownership changes. I would prove each wave with contract, parity, load, recovery, and security gates, and I would retain high-value mainframe components when that is the safer and more economical outcome.”

Ten facts worth remembering

1. Modernization ≠ mandatory migration off a mainframe.
2. The hardest problem is usually hidden domain and batch coupling, not API syntax.
3. Strangler routing changes *implementation* incrementally; it does not authorize dual writers.
4. ACL translates semantics; a gateway primarily handles common API concerns.
5. CDC creates read replicas and needs snapshot/log continuity, sequencing, and reconciliation.
6. One source of truth per bounded write domain prevents split brain.
7. A modern idempotency table alone cannot fix unknown commits inside legacy CICS.
8. Shadow reads are useful; production dual writes can create harmful side effects.
9. Read cutover is relatively reversible; write-owner cutover may not be.
10. Success means verified business behavior, reliability, security, and operating economics—not just moving COBOL to Java.

CHAPTER

17. Official technical references

1. **AWS — Strangler Fig pattern**: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/strangler-fig.html
2. **AWS — Anti-corruption layer pattern**: https://docs.aws.amazon.com/en_en/prescriptive-guidance/latest/cloud-design-patterns/acl.html
3. **IBM — z/OS Connect API providers**: https://www.ibm.com/docs/en/zos-connect/3.0.0?topic=connect-what-are-zos-api-providers
4. **IBM — Data replication on z/OS**: https://www.ibm.com/docs/en/zos-basic-skills?topic=tools-data-replication-zos
5. **IBM — CDC Replication for Db2 for z/OS**: https://www.ibm.com/docs/en/idr/11.4.0?topic=replication-infosphere-cdc-db2-zos
6. **AWS — Mainframe modernization approaches**: https://docs.aws.amazon.com/m2/latest/userguide/modernization-m2.html

7. **AWS — Mainframe decoupling best practices**: https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-mainframe-decoupling-patterns/best-practices.html
8. **NIST — SP 800-207 Zero Trust Architecture**: https://csrc.nist.gov/pubs/sp/800/207/final
9. **PCI Security Standards Council — PCI DSS v4.0.1 announcement**: https://blog.pcisecuritystandards.org/just-published-pci-dss-v4-0-1
10. **AWS — Mainframe cutover guidance**: https://docs.aws.amazon.com/prescriptive-guidance/latest/replatform-mainframe-apps-shared-db2/cutover.html

These references support the patterns and technology descriptions. The proposed architecture, example endpoints, entity schemas, migration gates, and operating requirements are design recommendations for the hypothetical scenario—not vendor guarantees or details of American Express's internal systems.